

SAGA Authorization Study: Paper Outline

Abstract

1 Introduction

1.1 Motivation

1.2 Problem

1.3 Key Observation

1.4 Contributions

2 Background and SAGA Overview

2.1 System Model

2.2 Agent Registration

2.3 Contact Policy

2.4 Agent Access Protocol

2.5 Access-Control Token

2.6 Existing Formal Security Claims

3 Security Gap and Problem Formulation

3.1 Authentication Is Not Authorization

SAGA uses identity checks and access-control tokens to govern contact between agents. These mechanisms answer different questions. Authentication establishes which participant completed a protocol step; token validation establishes that a presented token satisfies the checks performed by the receiving agent. Authorization additionally asks whether the receiving agent’s user-defined contact policy permits the initiating agent to contact it. A credential can be authentic and valid under the token checks while the decision to make that credential available disagrees with the policy’s intended meaning.

The reference communication model focuses on participant authentication and token secrecy. These guarantees concern the origin of protocol messages and the confidentiality of credentials; they do not, by themselves, establish that a contact complies with the receiving user’s policy. In particular, confidential delivery to an authenticated participant does not establish that the participant is entitled to receive a credential under that policy.

The missing requirement is an authorization basis for accepted contact, preserved through credential issuance and message acceptance. Chapter 4 examines implementation disagreements with policy meaning. This chapter defines the requirement without asserting an end-to-end access violation.

3.2 Policy–Authorization Consistency

We call the requirement *policy–authorization consistency*: every accepted access must be traceable to an applicable authorization decision. We apply it separately at each authorization boundary: releasing contact material, issuing a credential, and accepting a message. For every execution and every access event e , we require

$$\text{AccessAccepted}(e) \implies \exists d. \text{AllowDecision}(d) \wedge \text{Justifies}(d, e).$$

Here, d is a decision occurring in that execution, and $\text{AllowDecision}(d)$ means that it grants permission under the intended policy semantics. An implementation’s report of “allow” alone does not establish this predicate. The relation $\text{Justifies}(d, e)$ requires that the decision precede the accepted access, cover the same subject, target, and operation, and remain applicable under the relevant policy state. Where a credential mediates access, the decision must also be linked to the grant and credential used for that access. An

unrelated earlier allow decision is not a sufficient justification. This is a general requirement whose concrete events and policy-state conditions must be fixed for each formal model.

Let I be an initiating agent, R a receiving agent, and P_R the contact rulebook selected by R 's user. Each rule associates an agent-identity pattern with a budget. We write $\text{Allow}_{\text{spec}}(P_R, I)$ for the decision prescribed by the intended policy semantics, rather than the value returned by any particular implementation of the matcher. In the cases considered here, the unique highest-scoring matching rule determines the budget; a positive budget permits contact, whereas a nonpositive budget or no matching rule does not. Rule selection follows the matcher's documented most-specific-rule contract and its AID specificity score; the positive-budget convention follows the Provider's access check.¹ We initially fix the rulebook throughout an execution and consider either no matching rule or a unique highest-scoring match. Equal-score conflicts require an explicit convention and are outside this initial scope.

We distinguish three analysis-level events. $\text{ProviderGrant}(I, R)$ denotes the Provider's release, through its access interface, of the material needed for I to continue the contact protocol with R . The event $\text{TokenIssued}(R, I, T)$ denotes R 's creation of token T for I and its release for delivery through the protocol; it does not assert that I has received or used T . Finally, $\text{MessageAccepted}(R, I, T, m)$ denotes R 's acceptance of an application message m from I with token T , after the receiving agent's access checks, for application processing. It does not assert the completion of a downstream tool action.

Under this static-policy scope, two necessary conditions at the Provider and message-acceptance boundaries are

$$\text{ProviderGrant}(I, R) \implies \text{Allow}_{\text{spec}}(P_R, I),$$

$$\text{MessageAccepted}(R, I, T, m) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

The analogous requirement at token issuance is

$$\text{TokenIssued}(R, I, T) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

All three implications range over every occurrence of the respective events and all their arguments. They express permission at specific enforcement boundaries; they do not replace the decision-to-access linkage required by

¹See `match()` and `aid_specificity()` in `saga/common/contact_policy.py`, and `access()` in `saga/provider/provider.py`.

Justifies. Nor does policy permission alone suffice for access: certificate checks, key availability, token validity, expiry, and quota may still prevent a permitted contact. These conditions address contact permission, not quantitative bounds on credential issuance or message use.

For a policy that can change over time, the applicable policy version and the effect of revocation on existing credentials must additionally be specified. Quota consumption and token expiry likewise require explicit state and validity conditions. The general requirement leaves room for these obligations, but the static permission conditions above do not establish them.

A message-level authorization claim therefore requires a model connecting policy evaluation, credential issuance, and the final message-acceptance event. Chapter 6 develops the corresponding formalization and states its assumptions.

3.3 Authorization Pipeline

The relevant decision is distributed across several stages. The user's rulebook determines an intended answer for I contacting R . The Provider evaluates its stored copy of that rulebook and, if the request passes its checks, returns one-time key material. The initiating and receiving agents then complete the credential exchange; the receiving agent generates a token and later validates a token presented with a message. An authorization argument must connect the same initiating identity and receiving identity across these stages, rather than infer permission solely from the presence of a valid token. The intended progression is

$$P_R \longrightarrow \text{ProviderGrant} \longrightarrow \text{TokenIssued} \longrightarrow \text{MessageAccepted}.$$

The arrows indicate protocol stages and authorization dependencies, not a guarantee that reaching one stage causes the next to succeed.

Each boundary requires separate evidence: a Provider response does not establish token issuance, and token issuance does not establish message acceptance. Later checks and protocol steps may fail.

When $\neg \text{Allow}_{\text{spec}}(P_R, I)$ holds, we distinguish three levels of violation:

1. **Provider decision violation:** $\text{ProviderGrant}(I, R)$ occurs despite the policy denial.
2. **Credential issuance violation:** $\text{TokenIssued}(R, I, T)$ occurs despite the policy denial. Issuance alone does not establish delivery or subsequent use.

3. **End-to-end authorization violation:** $\text{MessageAccepted}(R, I, T, m)$ occurs despite the policy denial. This is acceptance at the receiving agent’s application boundary.

These levels classify possible observations rather than assert results. A sound pipeline must preserve the link to the applicable decision, participants, and credential across stages. Chapter 6 specifies the corresponding events; Chapter 7 evaluates implementation observations and model results separately.

3.4 Threat Model

We consider a normally registered initiating agent controlled by the adversary. It can choose its contact requests and participate in the protocol under its genuine identity and certificate. The receiving user’s rulebook is an existing configuration: the adversary cannot change its contents or rule order, control the Provider or CA, or obtain another honest party’s private keys.

The Provider and receiving agent execute their implementations with the same fixed rulebook. Cryptographic checks are assumed effective. The failure under study is a disagreement between policy meaning and enforcement during ordinary protocol operation, without compromise of either enforcement point.

The adversary’s objective is an execution in which the intended policy denies contact by I , yet the system performs one of the authorization-relevant actions above. At the message level, the violation condition is

$$\text{MessageAccepted}(R, I, T, m) \wedge \neg \text{Allow}_{\text{spec}}(P_R, I).$$

This end-to-end violation is distinct from an erroneous Provider grant or credential issuance. Claiming it requires evidence that the intervening checks and protocol steps reach the receiving agent’s acceptance boundary.

We study this objective under a static policy. Policy updates and revocation, disagreement between stored policy versions, quantitative exhaustion of budgets or quotas, compromise of trusted parties, and arbitrary actions after contact require additional assumptions and are outside this initial question. The distinction between zero, negative, and positive configured budgets remains part of the contact-permission semantics.

4 Authorization Inconsistencies in SAGA

4.1 Policy Matching Semantics

4.2 Rule-Specificity Implementation Flaw

4.3 Budget Semantic Inconsistency

4.4 Cross-Layer Authorization Inconsistency

4.5 Security Consequence

5 Attack Construction

5.1 Attack Goal

5.2 Attacker Capabilities

5.3 Rule-Ordering Attack

5.4 Zero-Budget Semantic Attack

5.5 Why Existing Cryptographic Protections Do Not Prevent the Attack

6 Formalization

6.1 Limitations of the Original Model

The registered initiating agent in Chapter 3 should not be silently identified with the attacker in the original symbolic model. That model provides an active network attacker over public communication and uses idealized cryptography, while its fixed peer processes execute their prescribed roles. Its authentication and token-secrecy queries do not include a contact rulebook or link a policy decision to credential use.

The Stage-1 authorization diagnostic preserves those assumptions and adds policy events without making policy evaluation an execution guard. In that diagnostic, a deny event is recorded but does not block the protocol, and the allow event is declared but never executed. The recorded failure of its added correspondence therefore identifies a missing modeled authorization dependence while the baseline authentication and secrecy results are retained. It does not by itself establish a matcher-dependent execution by a malicious registered agent in the implementation.

In particular, the diagnostic event named **Accept** denotes the initiating peer's successful receipt and verification of a token. It does not denote the receiving peer's acceptance of an application message. Its failed correspondence is thus evidence about authorization coverage at token receipt, not a violation of Chapter 3's message-level condition. A message-level claim requires an event at the receiving agent's message-acceptance point and a model connecting policy evaluation, credential issuance, and that event.

6.2 Authorization Events

6.3 Authorization Security Property

6.4 Vulnerable Model

6.5 Repaired Model

7 Evaluation

7.1 Research Questions

7.2 Implementation Validation

7.3 Formal Verification

7.4 Regression of Existing Properties

7.5 Performance and Compatibility

8 Discussion

8.1 Why Authentication Proofs Are Insufficient

8.2 Formal Verification Coverage

8.3 Policy Semantics as Security-Critical State

8.4 Generalization Beyond SAGA

8.5 Limitations

9 Related Work

9.1 Agent Security and Governance

9.2 Capability and Token-Based Authorization

9.3 Formal Verification of Authentication Protocols

9.4 Formal Verification of Authorization Policies

9.5 Authentication–Authorization Gaps

10 Conclusion